

1. Flatten a List<List<Integer>> into a single List<Integer>.

Explanation: You use the **flatMap()** operation. This transforms each inner list into a stream and then combines (flattens) all those individual streams into a single output stream. The **collect(Collectors.toList())** gathers the final result. **Solution:**

```
List<List<Integer>> nestedList = Arrays.asList(Arrays.asList(1, 2),
Arrays.asList(3, 4, 5));
List<Integer> flattened = nestedList.stream()
    .flatMap(List::stream) // Transforms each inner List into a single
    stream, then flattens
    .collect(Collectors.toList());
// Result: [1, 2, 3, 4, 5]
```

2. Group users by age into Map<Integer, List<User>>.

Explanation: You use the **collect()** terminal operation with the **Collectors.groupingBy()** collector. This method takes a function that extracts the grouping key (the age) and builds a map where the key is the age and the value is a list of all User objects that share that age.

Solution:

```
// Assuming a User class exists with a getAge() method
List<User> users = List.of(user1, user2);
Map<Integer, List<User>> usersByAge = users.stream()
    .collect(Collectors.groupingBy(User::getAge)); // Group by the age
    property
```

3. Find all numbers whose string representation starts with '1'.

Explanation: You use the **filter()** operation. Inside the filter, you convert the number to a string using **String.valueOf()** and then check if that string starts with the character '1'. **Solution:**

```
List<Integer> numbers = Arrays.asList(10, 25, 100, 321);
List<Integer> startsWithOne = numbers.stream()
    .filter(n -> String.valueOf(n).startsWith("1")) // Filter where
    string form starts with "1"
    .collect(Collectors.toList());
// Result: [10, 100]
```

4. Find all IDs that appear more than once (duplicates).

Explanation: You use **Collectors.groupingBy()** to count the frequency of each ID. Then, you filter the resulting map's entries to keep only those where the count is greater than 1, extracting just the key (the ID). **Solution:**

```
List<Integer> ids = Arrays.asList(1, 2, 1, 3, 4, 3);
```

```
Set<Integer> duplicates = ids.stream()
    .collect(Collectors.groupingBy(Function.identity(),
Collectors.counting())) // Group and count frequency
    .entrySet().stream()
    .filter(entry -> entry.getValue() > 1) // Filter entries where count
> 1
    .map(Map.Entry::getKey) // Extract the ID (the key)
    .collect(Collectors.toSet());
// Result: {1, 3}
```

5. Find the first non-repeated character in a String.

Explanation: You use **groupingBy** to count character frequencies. Then, you convert the stream of characters back from the original string. You use **filter()** to keep only characters whose count is 1 and use **findFirst()** to return the first one found. **Solution:**

```
String code = "aabbcd";
Character firstNonRepeated = code.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(),
Collectors.counting()))
    .entrySet().stream()
    .filter(entry -> entry.getValue() == 1) // Filter for count 1
    .map(Map.Entry::getKey)
    .findFirst() // Get the first one that appears in the stream
    .orElse(null);
// Result: 'c'
```

6. Find the first repeated character in that string.

Explanation: You use a **Set** to keep track of characters seen so far. You iterate through the string's characters, and the first time a character is found in the set, it is the first repeating character. You use the **filter()** and **findFirst()** operations. **Solution:**

```
String input = "abcda";
Set<Character> seen = new HashSet<>();
Character firstRepeated = input.chars()
    .mapToObj(c -> (char) c)
    .filter(c -> !seen.add(c)) // Filter: returns true if the add()
operation fails (i.e., it's a repeat)
    .findFirst()
    .orElse(null);
// Result: 'a'
```

7. Cube each price (price³) and then filter only those prices greater than 50.

Explanation: You use the **map()** operation to calculate the cube of each price. You then chain a **filter()** operation immediately after to keep only the results that are greater than 50.

Solution:

```
List<Integer> prices = Arrays.asList(2, 3, 4); // Cubes: 8, 27, 64
List<Integer> result = prices.stream()
    .map(p -> p * p * p) // Map: Cube the price
    .filter(p -> p > 50) // Filter: Keep only those greater than 50
    .collect(Collectors.toList());
// Result: [64]
```

8. Count how many times each word appears.

Explanation: This is a standard frequency counting exercise. You use **Collectors.groupingBy()** to group the words (using **Function.identity()** as the key) and use **Collectors.counting()** as the downstream collector to count the occurrences in each group. **Solution:**

```
List<String> words = Arrays.asList("apple", "banana", "apple");
Map<String, Long> wordCounts = words.stream()
    .collect(Collectors.groupingBy(Function.identity(),
    Collectors.counting()));
// Result: {apple=2, banana=1}
```

9. Find only those product names that are duplicates along with their count.

Explanation: You first group and count the names as in the previous example. Then, you filter the resulting map's entries to keep only those where the count (the value) is greater than 1.

Solution:

```
List<String> products = Arrays.asList("A", "B", "A", "C");
Map<String, Long> duplicatesWithCount = products.stream()
    .collect(Collectors.groupingBy(Function.identity(),
    Collectors.counting()))
    .entrySet().stream()
    .filter(entry -> entry.getValue() > 1) // Filter where count > 1
    .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue));
// Result: {A=2}
```

10. Safely check if a List<Object> is non-empty and then print each element.

Explanation: You wrap the list in **Optional.ofNullable()**. You use **.ifPresent()** to execute a consumer action only if the list is non-null. Inside the action, you check for emptiness and then use **.forEach()** on the list to print the elements. **Solution:**

```
List<String> list = dao.getData(); // Assume this might be null
Optional.ofNullable(list)
    .filter(l -> !l.isEmpty()) // Filter: Only proceed if list is not
    empty
    .ifPresent(l -> l.forEach(System.out::println)); // If present, print
    each element
```

11. Partition the numbers into even and odd and get a Map<Boolean, List<Integer>>.

Explanation: You use the **Collectors.partitioningBy()** collector. This collector takes a single Predicate (a boolean function) and automatically splits the input into two groups: true (even numbers) and false (odd numbers). **Solution:**

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4);
Map<Boolean, List<Integer>> partitioned = numbers.stream()
    .collect(Collectors.partitioningBy(n -> n % 2 == 0)); // Predicate:
    n is even
    // Result: {true=[2, 4], false=[1, 3]}
```

12. Calculate the frequency of each character and store it in Map<Character, Long>.

Explanation: You convert the string to an **IntStream** of character codes (**chars()**), map those codes back to **Character** objects, and then use **Collectors.groupingBy()** with **Collectors.counting()** to count the occurrences of each character. **Solution:**

```
String input = "testing";
Map<Character, Long> charCounts = input.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(),
    Collectors.counting()));
    // Result: {t=2, e=1, s=1, i=1, n=1, g=1}
```

13. Given a List<Integer> representing version numbers in order, verify if the list is already sorted in ascending order.

Explanation: You check if the list is sorted by comparing the original stream with a separate stream that has been explicitly sorted. If they are the same (element by element), the original list was already sorted. **Solution:**

```
List<Integer> versions = Arrays.asList(1, 2, 3);
boolean isSorted = IntStream.range(0, versions.size() - 1)
    .allMatch(i -> versions.get(i) <= versions.get(i + 1)); // Check if
    element[i] <= element[i+1]
```

```
// Result: true
```

14. You have a `List<List<String>>` where each inner list is a list of tags for a post. Using `flatMap`, how would you produce a single `List<String>` containing all tags?

Explanation: This is a direct use case for `flatMap()`. You transform each inner list of tags into a stream and flatten those streams into one single stream containing all tags. **Solution:**

```
List<List<String>> postTags = Arrays.asList(List.of("java", "stream"),
List.of("oop", "design"));
List<String> allTags = postTags.stream()
    .flatMap(Collection::stream) // Flattens all inner lists into a single
    stream of tags
    .collect(Collectors.toList());
// Result: ["java", "stream", "oop", "design"]
```

15. You are building pagination on top of a `List<Order>` using Streams. Given `pageNumber` and `pageSize`, how would you use `skip()` and `limit()` to get the desired page as a list?

Explanation: You calculate the number of elements to skip (offset) based on the page number. You use `skip()` to move to the starting element and `limit()` to restrict the result to the desired page size. **Solution:**

```
int pageNumber = 2; // 1-based index
int pageSize = 10;
long offset = (long) (pageNumber - 1) * pageSize;

List<Order> pagedOrders = allOrders.stream()
    .skip(offset) // Skip all elements from previous pages
    .limit(pageSize) // Take only the number of elements for the current
    page
    .collect(Collectors.toList());
```

16. You have a `Stream<String>` of words from multiple paragraphs. Using Streams, how would you count how many distinct vowels (a,e,i,o,u) appear across all words?

Explanation: You first use `flatMap` to break each word into a stream of its characters. You filter these characters to keep only vowels, use `distinct()` to ensure uniqueness, and finally use `count()` to get the total number of unique vowels. **Solution:**

```
Stream<String> words = Stream.of("Apple", "Banana", "Orange");
```

```
long distinctVowels = words
    .flatMapToInt(CharSequence::chars) // Get stream of char codes
    .mapToObj(c -> (char) Character.toLowerCase(c)) // Convert to
lowercase chars
    .filter(c -> "aeiou".indexOf(c) != -1) // Filter: Keep only vowels
    .distinct() // Count only unique vowels
    .count();
// Result: 3 (a, e, o)
```

17. You have a List<Integer> of IDs that you want to multiply together. Using Streams and reduce, how would you compute the product of all IDs?

Explanation: You use the **reduce()** terminal operation. You provide the identity (starting value, 1) and an accumulator (a function that multiplies two numbers). **Solution:**

```
List<Integer> ids = Arrays.asList(2, 3, 4);
int product = ids.stream()
    .reduce(1, (a, b) -> a * b); // Identity 1, Accumulator multiplies
elements
// Result: 24
```

18. You have a List<String> of city names. Using Streams, how would you sort them by their last character (custom Comparator) and return the sorted list?

Explanation: You use the **sorted()** operation and provide a custom Comparator that compares the last character of each string (s.charAt(s.length() - 1)). **Solution:**

```
List<String> cities = Arrays.asList("Delhi", "Mumbai", "Pune"); // Last
chars: i, i, e
List<String> sortedCities = cities.stream()
    .sorted(Comparator.comparing(s -> s.charAt(s.length() - 1))) //
Compares by last character
    .collect(Collectors.toList());
// Result: [Pune, Delhi, Mumbai] (or similar order if last chars match)
```

19. You have a List<String> of usernames that may contain null. Using Streams, how would you filter out all null values and return only non-null usernames?

Explanation: You use the **filter()** operation with a simple condition that checks if the element is not equal to null. **Solution:**

```
List<String> usernames = Arrays.asList("userA", null, "userB");
List<String> nonNullUsers = usernames.stream()
    .filter(Objects::nonNull) // Filter: Keep only non-null objects
    .collect(Collectors.toList());
// Result: ["userA", "userB"]
```

20. You have a `List<Integer>` of candidate numbers. Using Streams and a helper method, how would you filter the list to return only prime numbers?

Explanation: You use the **`filter()`** operation and pass a **method reference** (or a lambda) to a helper function (e.g., `isPrime`) that performs the primality check. **Solution:**

```
// Assume a static method 'MathUtils.isPrime(int n)' exists
List<Integer> candidates = Arrays.asList(4, 5, 6, 7);
List<Integer> primes = candidates.stream()
    .filter(MathUtils::isPrime) // Filter: Keep only numbers where isPrime
    returns true
    .collect(Collectors.toList());
// Result: [5, 7]
```